

Chapter 5: STACK (LIFO)

Stack is one of the most important containers in DSA.

Many famous interview questions use Stack:

- Balanced Parentheses
 - Next Greater Element
 - Undo Operation
 - Browser History
 - Expression Evaluation
 - Monotonic Stack Problems
-

What is Stack?

LIFO

Last In First Out

Think of plates:

```
Push 10
Push 20
Push 30
```

```
Top
↓
30
20
10
```

When removing:

```
Pop → 30
Pop → 20
Pop → 10
```

Last inserted comes out first.

Header File

```
#include <stack>
```

Declaration

```
stack<int> st;
```

Basic Operations

1. push()

Insert element.

```
st.push(10);  
st.push(20);  
st.push(30);
```

Stack:

```
30  
20  
10
```

Complexity

```
O(1)
```

2. pop()

Remove top element.

```
st.pop();
```

Before:

```
30  
20  
10
```

After:

```
20  
10
```

Complexity

```
O(1)
```

3. top()

Returns top element.

```
cout << st.top();
```

Output:

```
30
```

Complexity

```
O(1)
```

4. size()

Returns number of elements.

```
cout << st.size();
```

Output:

```
3
```

Complexity

```
O(1)
```

5. empty()

Checks if stack is empty.

```
if(st.empty())
```

Returns:

```
true  
false
```

Complete Example

```
#include <iostream>  
#include <stack>  
using namespace std;  
  
int main()  
{  
    stack<int> st;  
  
    st.push(10);  
    st.push(20);  
    st.push(30);
```

```
    cout << st.top() << endl;

    st.pop();

    cout << st.top() << endl;
}
```

Output:

```
30
20
```

Traversing Stack

✗ No iterators

✗ No indexing

You can only access:

```
top()
```

To print stack:

```
while(!st.empty())
{
    cout << st.top() << " ";
    st.pop();
}
```

Example

Input:

```
10 20 30
```

Output:

```
30 20 10
```

Copy Stack

```
stack<int> st2 = st;
```

Now both stacks contain same elements.

Swap Stack

```
st1.swap(st2);
```

Complexity

```
O(1)
```

Emplace()

Similar to push().

```
st.emplace(10);
```

Constructs object directly.

Mostly useful for classes.

Time Complexity Table

Operation	Complexity
push	O(1)
pop	O(1)

Operation	Complexity
top	$O(1)$
size	$O(1)$
empty	$O(1)$

Interview Question 1

Reverse a String

Input:

```
hello
```

Process:

Push:

```
h
e
l
l
o
```

Pop:

```
o
l
l
e
h
```

Output:

```
olleh
```

Code

```
string s;

cin >> s;

stack<char> st;

for(char ch:s)
{
    st.push(ch);
}

while(!st.empty())
{
    cout<<st.top();
    st.pop();
}
```

Interview Question 2

Balanced Parentheses

Input:

()()

Output:

Balanced

Input:

((()(

Output:

Not Balanced

Logic

For every:

(

push

For every:

)

pop

At end:

stack empty

=> Balanced

Code

```
bool balanced(string s)
{
    stack<char> st;

    for(char ch:s)
    {
        if(ch=='(')
            st.push(ch);

        else
        {
            if(st.empty())
                return false;

            st.pop();
        }
    }

    return st.empty();
}
```

Interview Question 3

Valid Parentheses

Input:

```
({[]})
```

Output:

```
Valid
```

Need matching brackets.

Uses:

```
(  
[  
{
```

and

```
)  
]  
}
```

Very common interview question.

Interview Question 4

Next Greater Element

Input:

```
2 1 5 3
```

Output:

5 5 -1 -1

Efficient solution uses Stack.

Complexity:

$O(n)$

instead of

$O(n^2)$

Monotonic Stack

Advanced Interview Topic.

Maintain stack in:

Increasing Order

or

Decreasing Order

Used in:

- Next Greater Element
- Previous Greater Element
- Largest Rectangle Histogram
- Stock Span Problem

Practice Questions

Easy

Q1

Push:

```
10 20 30
```

Print top.

Q2

Pop one element.

Print top.

Q3

Count stack size.

Q4

Print stack elements.

Q5

Reverse a word using stack.

Input:

```
world
```

Output:

```
dlrow
```

Medium

Q6

Check balanced parentheses.

Q7

Remove all elements from stack.

Q8

Find middle element of stack.

(Hint: use extra stack)

Q9

Copy one stack into another.

Q10

Insert element at bottom of stack.

Example:

Before :

30
20
10

Insert 5

After :

30
20
10
5

Interview Level

Q11

Valid Parentheses

LeetCode #20

Q12

Min Stack

LeetCode #155

Q13

Next Greater Element

LeetCode #496

Q14

Daily Temperatures

LeetCode #739

Q15

Largest Rectangle Histogram

LeetCode #84

Common Mistakes

Mistake 1

```
st.top();
```

on empty stack

✗ Runtime Error

Always:

```
if(!st.empty())
```

Mistake 2

```
st.pop();
```

on empty stack

✗ Runtime Error

Mistake 3

Trying:

```
st[0]
```

✗ Not allowed

No indexing.

Revision Sheet

Header

```
#include <stack>
```

Declaration

```
stack<int> st;
```

Insert

```
st.push(x);  
st.emplace(x);
```

Remove

```
st.pop();
```

Access

```
st.top();
```

Check

```
st.empty();  
st.size();
```

Swap

```
st1.swap(st2);
```

Complexity

```
push   -> O(1)  
pop    -> O(1)  
top    -> O(1)  
size   -> O(1)  
empty  -> O(1)
```

Real Interview Insight

When you see:

- Parentheses
- Undo/Redo
- Backtracking
- Browser History
- Next Greater Element
- Expression Evaluation

Your brain should immediately think:

```
STACK
```

Assignment

Solve these before moving on:

1. Reverse string using stack.
2. Balanced Parentheses.
3. Valid Parentheses.
4. Insert at bottom of stack.

5. Reverse entire stack.
 6. Next Greater Element.
 7. Stock Span Problem.
 8. Min Stack.
-